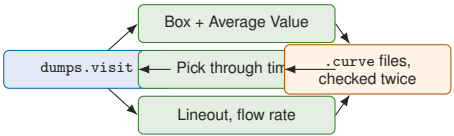
VISIT: DATA ANALYSIS

Sampling, averaging and measuring: a box, queries frame by frame and through time, a pick through time, lineouts, the flow rate across a line, expressions, curves exported and read back; and a written record of how every number was made



pictures and numbers: VisIt 3.5.0 on IB/11 (the heart-tube racetrack, the 2019 lecture's sample geometry); windows: the 2019 deck

Laura A. Miller

Department of Mathematics, University of Arizona

Using IB Software • Part V • Chapter 29

Claude (Anthropic) was used to organize, combine, and format more than fourteen years (2012–2026) of the instructor's tutorial notes from courses and workshops at several universities.

LEARNING GOALS

By the end of this chapter you should be able to:

- Keep a written record of an analysis that someone else could repeat: a protocol, a data sheet per run, a report.
- Restrict a measurement to a box, and average a field over it, frame by frame and through time.
- Follow a variable through time at a point (a pick through time), and sample it along a line (a lineout).
- Turn a lineout into a flow rate, and check a flow rate on several lines.
- Export curves from VisIt and read them in Python, MATLAB or R.
- Check every number a second way, because some of VisIt's routes give different answers.

> Every number in an analysis is a box, a point or a line, a query, and a time axis. Write all four down, and measure each number two ways.

WHERE THIS CHAPTER COMES FROM

- Lindsay Waldrop's VisIt guide, "Data analysis in VisIt" (MATH 769; dated 16 February 2015 and written with VisIt 2.5.2 for IBAMR's output; quoted here from its 2017 version for IB2d's `.vtk` output): loading data, plots, movies, and a "Data Analysis" section: best practices for data analysis, restricting data capture with a box, Query and Query over time, a pick at a point through time, exporting curves (MATLAB, R).
- "4 – Visit Data Analysis – Sampling and Time Queries" (2019, 24 slides, "Adapted from Lindsay Waldrop's Visit Tutorial"): "the first of three lectures on how to extract information from data in visit" (the GUI; export to MATLAB; Python on the cluster); the same recipes on a racetrack pumped by an elastic section, plus a lineout.
- "Lecture 9: Sampling, averaging, and analyzing data in Visit" (59 slides, "Adapted from Lindsay Waldrop's and Nick Battista's Visit Tutorials"): slides 37–59 are the 2019 deck's text (its first 36 slides are chapters 26 and 27).
- "3B – Analyzing your IBFE simulation" (2018), slides 12–16: a box above the corals, its average vertical flow through time, the curve in MATLAB.

New here: the 2019 sample data is IB/11's racetrack (next slides), so every recipe is redone on it in VisIt 3.5.0 and checked: the time query and the frame-by-frame query compared, the lineout turned into a flow rate on six lines round the loop, the curve files read in Python and in MATLAB/Octave (R's line given, not run), templates for the written record, and a script for all of it (`examples/analysis/visit_queries.py`).

BEST PRACTICES FIRST (WALDROP)

"As researchers, we are responsible for leaving a clear, understandable record of what we did to reach the conclusions in our papers . . . transparency and reproducibility are the ultimate goals of all scientific endeavors."

A written protocol should include:

- the coordinates of each box or position where data were collected or analysed, with a labelled figure;
- the method(s) by which the data were analysed ("were the data in the box averaged? was the max value found?");
- step-by-step instructions for reproducing each output file, "detailed enough that an undergraduate with a basic familiarity with VisIt could reproduce your plots without consulting you directly";
- any plots, and the code ("documented appropriately") for reproducing them.

And a data sheet in each simulation's folder: "a clear, handy record of the parameters that the simulation used and any other identifying information".

L. Waldrop, VisIt guide, "Best Practices for Data Analysis".

THE ANALYSIS REPORT, AND GENERAL TIPS (WALDROP)

An analysis report: a condensed protocol ("what you would write in the Materials and Methods section of a paper"), any statistics so far, graphs with thorough captions, and rough notes on what you found; "an excellent first draft of subsequent writing".

General tips:

- "Err on the side of writing down too many details." "Record as you go along."
- "Pick a clear naming convention and stick with it": for a sweep, one folder per sweep and one subfolder per value ("Wosweep" holding "Wo10", "Wo0.5"), and a spreadsheet of where everything is.
- "Leave a READ ME file in the first directory": the study, the naming system, where the spreadsheet, the protocol and the report are, and which runs exist but were not used.
- "Clean up old files and simulations as you go along."
- "Pretend like you won't remember anything in a week."

New: `examples/analysis/templates/`: `analysis_protocol.md`, `data_sheet.md`, `analysis_report.md`, each with this chapter's IB/11 analysis filled in as the example (the guide says examples "should be included with these files"; none came with the copy used here).

L. Waldrop, VisIt guide, "Best Practices for Data Analysis".

THE SAMPLE DATA IS IB/11'S RACETRACK

The 2019 deck: "The data was generated from a code where flow is driven through a racetrack through the pumping of an elastic section at the bottom of the track. The sides and the top of the racetrack are rigid. There is also a rigid box around the elastic part. If you plot markers, it will show passive tracers … If you plot `vp_aforce` it will show the springs that drive the contraction."

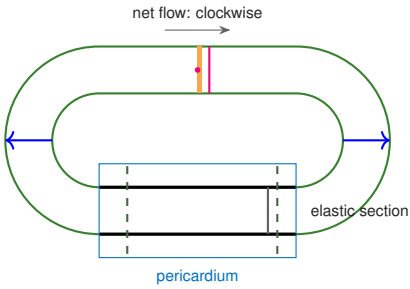
That is IB/11 (chapter 9b): the loop's centre lines at $y = \pm 0.15$, tube diameter $D = 0.1$, the elastic bottom section inside a rigid pericardium. The deck's box ($0 \le x \le 0.01$, $0.10 \le y \le 0.20$), point $(0, 0.15)$ and line $(0, 0.1)$–$(0, 0.2)$ all sit on the top vessel.

Here: the `TARGET_VP` run of chapter 9b (suction by target points; IBAMR 0.19.0): 51 frames, $t = 0$ to 2 s every 0.04 s, 4 levels, finest $\Delta x = 1/512$; $\rho = 1000$, $\mu = 0.157$ (Wo $= 10$ at 1 Hz). The flow round the loop also uses `FORCE_VP` (suction by an applied force, `Fmag_vp` $= 0.25$).

IBAMR 0.19 writes the velocity only as the vector `U` (the 2019 menu had `U_x`): define `U_x = U[0]`, `U_y = U[1]`, `speed = magnitude(U)` in **Controls** → **Expressions** (chapter 26), which also avoids VisIt 3.5's trouble with its own `U_magnitude` beside a plot from a second database (chapter 27).

The 2019 deck, slide 3; `examples/IB/11-heart-tube-pump` and its README.

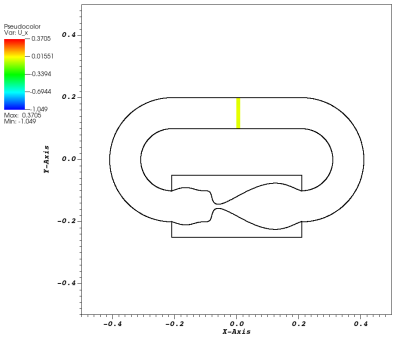
WHERE WE MEASURE



- Box: $0 \le x \le 0.01$, $0.10 \le y \le 0.20$, a stripe across the top vessel (the 2019 deck's).
- Point $(0, 0.15)$, the vessel's centre, and the line from $(0, 0.1)$ to $(0, 0.2)$ across it (drawn beside the box here; it lies on the box's edge).
- The flow rate round the loop on six lines: the top line, the two bends at $y = 0$ (blue), the elastic section at $x = 0.15$ between the walls' rest positions (grey), and at $x = \pm 0.15$ across the whole pericardium (dashed, drawn apart).
- Positive flow on every line: clockwise round the loop, $+x$ along the top.

Drawn to scale from IB/11's generator (chapter 9b).

RESTRICTING DATA CAPTURE WITH A BOX

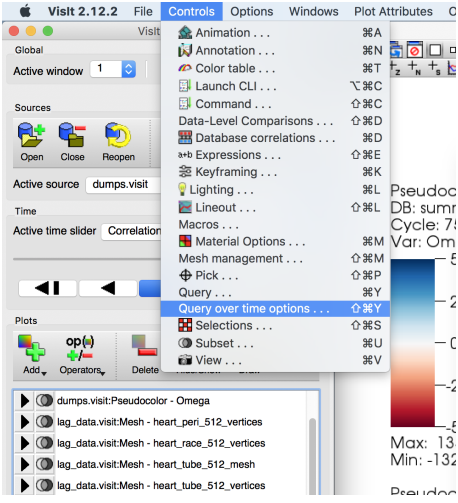


"It is sometimes useful to restrict your data analysis to a very small part of your total graph … It doesn't matter how wide the box is, so long as you use the same box for every step of the analysis (and every simulation you analyze)."

- "Under the Plots section, you should see two check boxes labeled 'Apply operators to all plots' and 'Apply subset selections to all plots.' Uncheck both of these boxes." Then a Pseudocolor plot of `U_x`, **Operators** → **Selection** → **Box**, and the box's minimum and maximum x and y (the window also has z fields, 0 to 1 by default, as the 2019 deck's window on its slide 9 already had: keep a range containing 0, chapter 28).
- Picture: `U_x` only inside the box at $t = 2$ (one yellow: the colour bar keeps the whole field's range, -1.05 to 0.37 m/s, while `U_x` in the box runs from -0.039 to 0.003), and the loop's points.
- **Amount of cell in the range**: **Some** keeps every cell that is partly inside or touches the box (here 7 × 52 cells of $1/512$ m: a column that only touches $x = 0$, and rows over 0.1016 m for the vessel's 0.1), **All** only cells wholly inside (5 × 50). Here, on the four frames checked, **Some** kept 0.0014–0.0018 m^2 for the nominal 0.001, changing as the grid regridded; **All** 0.00088–0.00095. Say which you used.

The 2019 deck, slides 5–9 (the guide's 2017 version boxes a quarter of IB2d's rubber band; its 2015 original used $0 \le x \le 0.01$, $0.10 \le y \le 0.12$, where "the vessel goes from about 0.10 to 0.12" in its geometry). IB/11 `TARGET_VP`, VisIt 3.5.0; areas from the 2D area query.

QUERY OVER TIME: THE TIME AXIS

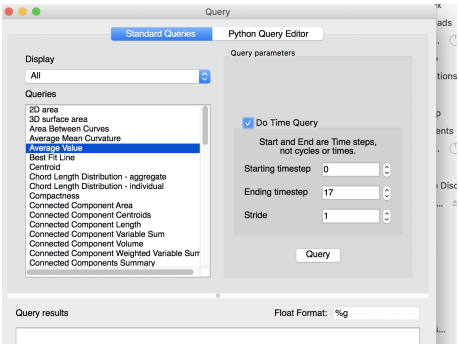


“Before your first Query, you need to select ‘Query over time options. . . ’ right below Query . . . there will be a line of toggle buttons along the top with the options ‘Cycle, Time, Timestep.’ Select ‘Time’ and hit Apply.”

- The choice is the curve’s x-axis: the cycle (time-step number), the time, or the frame index. VisIt 3.5’s default is still **Cycle** (in a script: `GetQueryOverTimeAttributes().timeType`, with the values `Cycle`, `DTime`, `Timestep`).
- The same window says where the curve goes: **Use 1st unused window or create new one** (ticked by default; window 2 here), or a **Window #** you give. The frames (start, end, stride) are chosen in the Query window (next slide’s picture): the Query over time window lost them in 2011, though the 3.5 manual and `GetQueryOverTimeAttributes()` still list them.
- Queries that run through time in 3.5 include **Average Value**, **Variable Sum**, **Weighted Variable Sum**, **Min**, **Max**, **2D area**, **Volume** and the two trajectory queries (`QueriesOverTime()` lists 20).

Left: the 2019 deck, slide 11 (VisIt 2.12.2). VisIt 3.5.0: `QueriesOverTime()`, `GetQueryOverTimeAttributes()`, the window’s source (`QvisQueryOverTimeWindow.C`).

AVERAGE VALUE THROUGH TIME

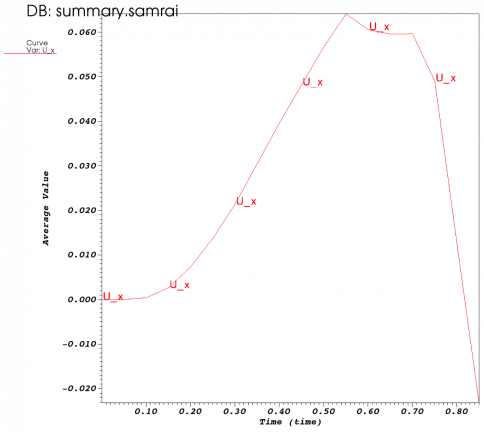


“Next, select Query. In this box, there is a list of options on the left that is long . . . we’ll select ‘Average Value’ . . . select the checkbox that says ‘Do Time Query’ . . . Hit the ‘Query’ button and wait for a graph.”

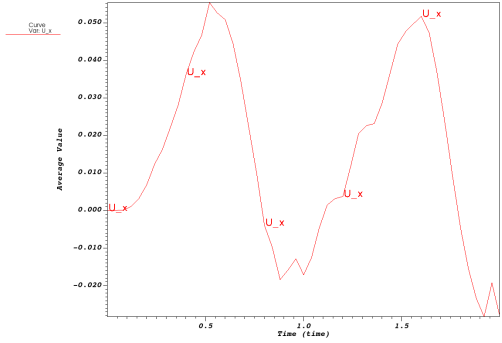
- The 2019 deck: several queries go onto one graph, another variable (`U_y`, the speed) or another query (Min, Max): “Each line should show up as a different color” (not tried in 3.5).
- **Average Value** is the area-weighted mean of the cells in the box: on the four frames checked it equals **Weighted Variable Sum** divided by **2D area** to the last digit.
- Next slide: the 2019 result and VisIt 3.5’s.

The 2019 deck, slides 12–13 (VisIt 2.x). The equality: IB/11, VisIt 3.5.0, frames 1, 10, 25, 50.

THE CURVE, THEN AND NOW



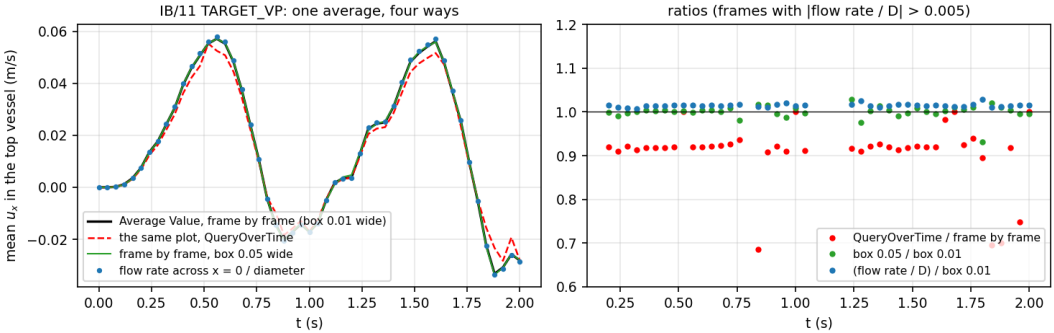
2019: the average of `U_x` over the box through the frames of the deck's sample data ("For the limited data set I provided, your graph will look like this").



VisIt 3.5.0: the same query through time on IB/11 `TARGET_VP`, 51 frames, time on the x-axis: two pumping periods. The next slides check it.

Left: the 2019 deck, slide 15. Right: `visit_queries.py`.

ONE AVERAGE, FOUR WAYS



Left: the box average of `U_x` on IB/11 `TARGET_VP`, frame by frame (black), through time on the same plot (red dashes), frame by frame in a box 5 times wider (green), and the flow rate across the box's line divided by the diameter (blue dots). Right: the ratios, on the frames where the mean speed exceeds 5 mm/s.

`visit_queries.py` (VisIt 3.5.0) and `tools/ch29_figures.py`.

WHAT THE COMPARISON SAYS

- The time query is low. On the boxed plot, **Query over time** gave 0.919 of the frame-by-frame value (median over the frames with a mean speed above 5 mm/s; 0.69–0.75 on four frames of reversed flow, $t = 0.84$, 1.84, 1.88 and 1.96; exactly 1 at $t = 0.52$, 1.0, 1.68 and 2.0 in this run, but 0.91 and 0.92 at $t = 1.0$ and 2.0 in another session's time query). The same comparison without the Box gave a ratio of 1.0000 on all four frames compared: the trouble is the Box in a time query on this AMR data, in VisIt 3.5.0. With **All** instead of **Some** the time query skipped four frames altogether.
- The frame-by-frame value is right. It agrees with the flow rate across the box's left edge divided by the diameter to within 2.9 % on every frame with a mean speed above 5 mm/s (the flow along the straight top vessel is nearly the same at $x = 0$ and $x = 0.01$; the median is 1.5 % above the box average, because the **Some** box spans 0.1016 m, a little beyond the walls, where the flow is zero); the time query is 8 % off.
- The box width matters a little. A box 5 times wider ($0 \le x \le 0.05$) changed the average by up to 6.9 % on the frames with a mean speed above 5 mm/s (by at most 0.001 m/s on any frame): hence the 2019 rule, the same box for every step and every run, and its coordinates in the protocol.

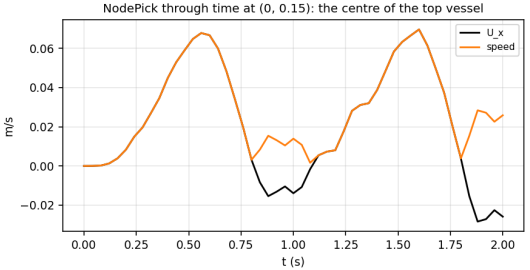
> Check a time query against the same query on a few single frames (move the time slider, **Query**, compare). If they differ, loop over the frames yourself: `visit_queries.py` does.

IB/11 `TARGET_VP`, 51 frames; VisIt 3.5.0 without a window (the GUI was not tried); `wsl/final/visit_ch29_avg*.sh`.

A PICK THROUGH TIME AT A POINT

The 2019 recipe: “Be sure that ‘Time’ is selected … Open the Query Window. Under ‘Display’ menu, select ‘Variable-related.’ Then, under Queries, select ‘Pick.’ Set your variable … Enter in your coordinates for single point … Let’s set ours to 0 0.15 0. Select ‘Pick using coordinate to determine zone’ … ‘Preserve Picked Coordinate’ … ‘Create Single Y-Axis plot.’ Check the box labeled ‘Do Time Query’ … Hit Query.”

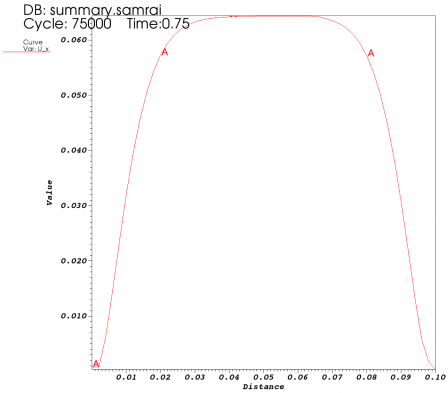
- “If you use the drop-down box, be sure to manually erase the previous variable (or the word ‘default’) because selecting a new variable will not erase the last one used.”
- Zone or node. The 2019 deck picks by zone, as Waldrop’s 2015 original (for IBAMR) did; the IB2d version of her guide changed it to “coordinate to determine node”. In VisIt 3.5 a zone pick through time on `U_x`, which is node-centred in IBAMR 0.19’s output, refuses: “The centering of the pick-through-time (zone) does not match the centering of the plot’s current variable (node). Please try again with the appropriately centered Pick” (VisIt’s words). Use the node pick.



A node pick through time at $(0, 0.15)$ of `U_x` and the speed, on one y-axis: u_x peaks at 0.070 m/s ($t = 1.6$) and reverses to -0.028 ($t = 1.88$); where it is negative the speed is its absolute value.

The 2019 deck, slides 16–18; Waldrop’s guide. IB/11 `TARGET_VP`, VisIt 3.5.0 (`NodePick(coord=..., do_time=1, preserve_coord=1, curve_plot_type=0)`).

A LINEOUT: A VARIABLE ALONG A LINE AT ONE TIME

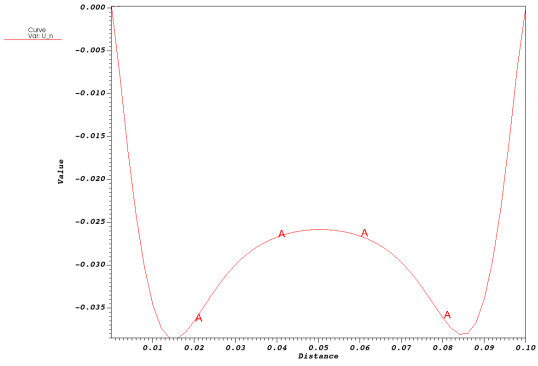


The 2019 lineout across the vessel (VisIt 2.x, $t = 0.75$): a flat core and steep walls.

"If you want to analyze data along a line in your plot area . . . under Queries, select 'Lineout.' . . . Enter in your coordinates for the beginning and end of the line . . . Let's set ours to 0 0.1 0 and 0 0.2 0. Click Use Sampling. The default number of 50 is typically pretty good. Hit Query."

- A lineout is one frame. For a lineout in every frame, loop over the frames (slide 26).

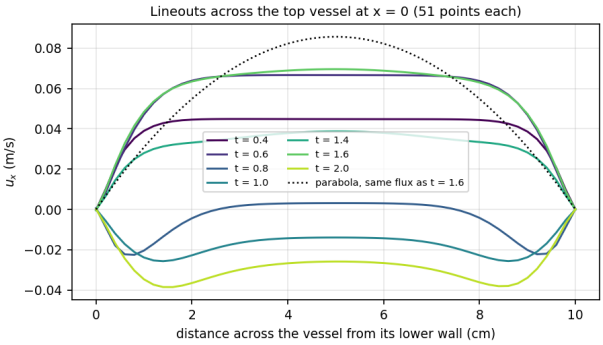
THE LINEOUT IN VISIT 3.5, AND SAMPLING



- The same line on `TARGET_VP` at $t = 2$ in VisIt 3.5 (the variable is `U_n`, the velocity across the line, slide 18: here `U_x`), while the flow reverses: the fluid near the walls moves backwards faster than the core (slide 17).
- With sampling, 50 samples give 51 points, equally spaced. Without it, VisIt takes the value wherever the line crosses a cell: 52 points here ($0.1/\Delta x = 51.2$ cells); on a coarse level, few.

IB/11 `TARGET_VP`, VisIt 3.5.0 (`Lineout(..., use_sampling=1, num_samples=50)`).

THE PROFILE ACROSS THE VESSEL THROUGH A PUMPING PERIOD



- Lineouts across the top vessel at $x = 0$, one in every frame (50 samples, 51 points each): seven of them shown.
- Not parabolas. At $t = 1.6$ the profile is a flat plug with a centre speed of 0.070; a parabola with the same flow rate would peak at 0.086 (dots).
- When the flow reverses ($t = 0.8$, 2.0), the fluid near the walls turns first: at $t = 2.0$ the profile's extreme, -0.0385, lies 16 mm from the wall, and the centre has only -0.026.
- Both are oscillatory flow at a Womersley number of 10 (chapter 8b): viscosity reaches only a thin layer at the wall in one period, and the core moves as a plug.

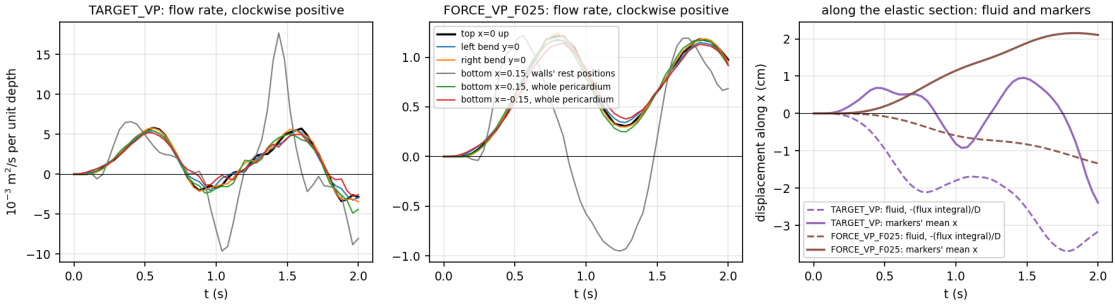
IB/11 `TARGET_VP`; `visit_queries.py` (`lineouts.csv`), `tools/ch29_figures.py`.

FROM A LINEOUT TO A FLOW RATE

- The flow rate per unit depth across a line is $Q = \int \mathbf{u} \cdot \mathbf{n}\, ds$. On the vertical line $(0, 0.1)$–$(0, 0.2)$, $\mathbf{u} \cdot \mathbf{n} = u_x$.
- For any line, define the velocity across it: with the line from (x_0, y_0) to (x_1, y_1), length L, the normal to its right is $\mathbf{n} = (y_1 - y_0,\ -(x_1 - x_0))/L$, and `U_n = n_x*U[0] + n_y*U[1]` with the numbers put in (**Controls** → **Expressions**).
- Lineout of `U_n` with sampling, then **Query** → **Integrate** on the curve in window 2: the area under it. At $t = 2$ on `TARGET_VP`: -0.0028609 m^2/s, the same as the trapezoid rule on the 51 points.
- Through time: one lineout per frame, integrated (`visit_queries.py` writes `averages.csv` with `flow_rate_line` for every frame).

> A box average and a flow rate answer the same question in a straight vessel; on a moving or curved part they do not, and a line is easier to place.

THE FLOW ROUND THE LOOP, ON SIX LINES



Left and middle: the flow rate on the six lines of slide 7, clockwise positive, for the two suction modes of IB/11. Right: along the elastic section, how far the fluid moved (minus the time integral of the top line's flow rate, divided by D; dashed) and how far the markers' mean x moved (solid).

`visit_queries.py` on six lines per run (`wsl/final/ch29_loopflux*.sh`); the markers' mean x from `lag_data.visit` (**Variable Sum** of their x over 121 points).

WHAT THE SIX LINES SAY

line (mean over 0–2 s, m^2/s per unit depth)	TARGET_VP	FORCE_VP
top straight, $x = 0$	1.582×10^{-3}	6.688×10^{-4}
left bend, $y = 0$	1.578×10^{-3}	6.675×10^{-4}
right bend, $y = 0$	1.578×10^{-3}	6.687×10^{-4}
elastic section, $x = \pm 0.15$, whole pericardium	$1.42, 1.69 \times 10^{-3}$	$6.55, 6.60 \times 10^{-4}$
elastic section, $x = 0.15$, walls' rest positions	1.20×10^{-3}	2.51×10^{-4}

- The fluid is incompressible, so the flow round a closed loop is one number: the top and both bends agree to 0.3 %. Both suction modes turn the loop clockwise, $-x$ along the elastic section. TARGET_VP reverses within each period; FORCE_VP never does.
- Across the elastic section only a line over the whole pericardium comes within 10 % (its two lines give 1.42 and 1.69×10^{-3}, and 6.55 and 6.60×10^{-4}); a line between the walls' rest positions misses the moving tube.
- The markers are not the flow. The FORCE_VP markers drift $+2.1$ cm in two periods while the fluid along the elastic section moves -1.3 cm (slide 19); 22 of the 121 markers start on the walls' rest positions and ride with the walls. Chapter 9b read the markers as the flow; it is corrected.

The numbers: tools/ch29_figures.py on the six flux_*.csv files of each run.

THE SAME FLOW RATE FROM A READER OF YOUR OWN

- Chapter 33 computed the flow rate of `FORCE_VP` across the top straight at $x = \pm 0.15$ with its own reader of the SAMRAI files (`examples/analysis/flow_rate.py`): means 6.64 and 6.65×10^{-4} m^2/s.
- VisIt's lineouts on the line at $x = 0.15$, $y \in [0.1, 0.2]$, integrated frame by frame: a mean of 6.656×10^{-4} over the 51 frames (6.69×10^{-4} as a time integral). Two independent routes agree to 0.1 % (with chapter 33's 6.65, given for $x = 0.15$; both plain means over the frames).
- That is the habit this chapter asks for: every number from two routes that share nothing but the data.

> A number measured once is a guess with a decimal point. A number measured two ways that agree is a result; two ways that disagree are a finding (slides 13 and 20).

Chapter 33, slide "The flow rate of the valveless pump"; `visit_queries.py` with `line=0.15,0.10,0.15,0.20`.

EXPORTING CURVES

“At the top of the main window, select Active Window: 2 or whichever window your plot is in. Highlight the curve . . . under ‘File,’ select ‘Set Save Options. . . ’ . . . select the file type ‘curve.’ . . . Change the file name and output directory. Click ‘Save.”’

- “VisIt outputs data as a two-column text file with a short header, delineated by spaces (instead of commas, like a csv file).”
- VisIt 3.5 writes a line `# name` before each curve, then one `x y` pair per line; a pick of two variables through time gives two curves in one file. In a script, `SaveWindow` with `format = CURVE` on the curve window (it kept the name and folder given, unlike the VTK export of chapter 33).

```
# U_x
0 0
0.03999999910593033 2.368893728998955e-05
0.07999999821186066 0.0001810442772693932
...
# speed
0 0
0.03999999910593033 3.270131855970249e-05
...
```

The pick through time of slide 14 as VisIt wrote it (shortened).

The 2019 deck, slides 22–23; Waldrop’s guide. VisIt 3.5.0.

READING CURVES: PYTHON, MATLAB, R

```python
# Python: examples/analysis/read_curve.py
from read_curve import read_curve
c = read_curve("pick.curve")   # {'U_x': (t, u), 'speed': (t, s)}
t, u = c["U_x"]
# one curve per file: numpy.loadtxt(path, comments="#")
```

```
% MATLAB or Octave: examples/analysis/read_curve.m
c = read_curve('pick.curve');   % c(k).name, c(k).x, c(k).y
plot(c(1).x, c(1).y)
% one curve per file (the 2018 lecture):
d = importdata('average.curve');  t = d.data(:,1);
```

```
# R (Waldrop's guide)
data <- read.table("visit10000.curve", header=FALSE, sep="")
```

- The 2018 lecture's `importdata`: "time is the first column . . . velocity is the second"; Octave 8.4 returned a struct whose `data` is 51×2 for the average's file.
- For a file with two curves (the pick), `importdata` and `loadtxt` run them together (Octave 8.4's `importdata`: 103×2, an `NA` row where the second name was; `loadtxt`: 102×2); `read_curve` keeps them apart. Both readers were run on this chapter's files.
- Waldrop's R line gives two columns `data$V1` (time) and `data$V2` (the variable); its `sep=""` means any white space. R was not run here.

The 2018 lecture "3B – Analyzing your IBFE simulation", slide 16; Waldrop's guide.

THE IBFE VERSION: A BOX ABOVE THE CORALS (2018)

"Graphing spatially averaged vertical flow. Make a pseudocolor plot of `U_y`. Use **Operators** → **Selection** → **Box** to average in a box. Draw a box above the corals of whatever size you like. You must specify range in x, y and z. What you pick does not matter too much as long as it's consistent."

- Then the same time query (**Query over time options**: time; **Average Value**, **Do Time Query**) and the curve saved for MATLAB.
- The 2018 deck did it on a cluster whose VisIt could not read `.ex2` files, so the coral meshes went through VTK (chapter 28); the fluid's box average needs only `dumps.visit`.
- "What you pick does not matter too much as long as it's consistent": true for comparing runs, but a box 5 times wider changed the IB/11 average by up to 7 % (on frames with a mean speed above 5 mm/s); report the box with the number.
- In 3D the box is a volume, **Average Value** its volume-weighted mean, and the time query deserves the same check as in 2D (slide 13).

The 2018 lecture "3B – Analyzing your IBFE simulation", slides 12–16 (VisIt 2.12).

EXPRESSIONS FOR ANALYSIS

name	definition	what for
`U_x`, `U_y`	`U[0]`, `U[1]`	components (IBAMR 0.19 writes the vector only)
`speed`	`magnitude(U)`	instead of VisIt's `U_magnitude` beside a plot from a second database
`U_n`	`nx*U[0] + ny*U[1]`, the line's numbers in	the velocity across a line, for a flow rate
`vortmag`	`magnitude(Omega)`	3D vorticity magnitude (chapter 27)
`dX`	`{dX_0, dX_1, dX_2}`	an IBFE body's displacement in 3D (VisIt builds the 2D one itself; chapter 28)

- An expression is computed when a plot or query asks for it, frame by frame, and saved in the session, not in the data (chapter 26).
- VisIt's expression language has the usual functions (`abs`, `sqrt`, `min`, `max`, `if`), vector operations (`dot`, `cross`, `curl`, `gradient`) and coordinates (`coord(mesh)`): the markers' mean x of slide 19 was `coord(markers_512_vertices)[0]`, summed with **Variable Sum** and divided by 121.

ALL OF IT, SCRIPTED

```
visit -nowin -cli -s visit_queries.py -- \
      viz_IB2d/dumps.visit out \
      box=0,0.01,0.10,0.20 point=0,0.15 \
      line=0,0.10,0,0.20 samples=50
```

`examples/analysis/visit_queries.py` runs inside VisIt's Python without a window (about 30 s for IB/11's 51 frames) and writes:

- `ch29-box.png`; the curve windows as PNG and `.curve`: the average through time, the pick through time, the lineout (sampled; the unsampled one as `.curve` only);
- `averages.csv`: per frame, the box average (frame by frame) and the flow rate across the line;
- `lineouts.csv`: every frame's lineout.

```
for k in range(N):                          # every frame
    SetTimeSliderState(k)
    Query("Average Value")                  # on the boxed plot
    avg = GetQueryOutputValue()
    Lineout(start_point=(0, 0.1, 0), end_point=(0, 0.2, 0),
            vars=("U_n",), use_sampling=1, num_samples=50)
    SetActiveWindow(2)                      # the lineout's
  window
    xy = GetPlotInformation()["Curve"]      # x0, y0, x1, y1,
  ...
    DeleteAllPlots(); SetActiveWindow(1)
```

The loop that replaces the time query (simplified: the script runs two loops, and makes the lineouts on a plot of `U_x` without the Box). Chapter 32 is about VisIt's Python; chapter 31 runs it on the cluster.

WHEN THE ANALYSIS GOES WRONG

what you see	what to do
the box changes every plot, or the structure disappears	untick “Apply operators to all plots” (and “. . . subset selections . . . ”) before adding the Box
`U_x` is not in the menu	IBAMR 0.19 writes `U` only: define `U_x = U[0]`
the time axis shows cycles	**Query over time options**: **Time**
a time query disagrees with single frames	check a few frames by hand; loop over frames (slide 13)
a time query skips frames	seen with the Box set to **All**: use **Some**, or loop over the frames
a scripted time query on the boxed plot gives no curve (“The Curve plot of variable U_x yielded no data”)	it had `use_actual_data=0`: use `1`
“The centering of the pick-through-time (zone) does not match . . . ”	pick by node
the pick follows the old variable	erase the previous variable (or “default”) in the box
a lineout has few, uneven points	tick **Use sampling**
a flow rate has the wrong sign	the line’s direction: positive flow crosses it from left to right
the flow rate across a moving part is small	the line misses the structure: span the whole region (slide 20)
the markers say one thing, the flow rate another	believe the flow rate; markers are a few parcels
`importdata` runs two curves together	`read_curve`

EXERCISES

1. Fill in `templates/analysis_protocol.md` for one measurement of your own, and ask someone to repeat it from the protocol alone.
2. Repeat slide 12 by hand in the GUI: the box, **Average Value** on three frames, then the time query. Do the two agree in the GUI?
3. Run `visit_queries.py` on `FORCE_VP` with the default box, point and line. How does the pick through time differ from `TARGET_VP`'s?
4. Measure the flow rate of `TARGET_PERI` (the target wave) on the top straight and on both bends. Which way does it pump, and do its markers agree?
5. The guide's own example: IB2d's `Rubberband_with_Springs`, `Omega` in the box $0.5 \le x \le 1$, $0 \le y \le 0.5$, its average through time, and a node pick at $(0.75, 0.25)$ (chapter 4a runs it).
6. Make the lineout of slide 17 at the moment of largest flow and fit a parabola to the inner half: how far from Poiseuille is it?
7. Read one of your `.curve` files in R with the guide's `read.table` line, and in Python with `read_curve`.

SUMMARY

- Write the analysis down as you go: a protocol with every box, point and line, a data sheet per run, a report (templates in `examples/analysis/templates`).
- A Box restricts a plot; **Average Value** is the area-weighted mean over its cells (**Some** or **All**: say which).
- **Query over time** with **Time** as the axis; on IB/11 with a Box it was 8 % low against the same query frame by frame, so check it or loop over the frames.
- A node pick through time follows a point; a lineout with sampling samples a line; integrated, it is the flow rate across the line.
- The flow round IB/11's loop is one number on every line across its rigid parts (to 0.3 %), and close to it on a line across the whole pericardium, clockwise in both suction modes; the markers' drift is not that number.
- Curves leave VisIt as `.curve` text and come back with `read_curve` (Python, MATLAB/Octave), `importdata`, `loadtxt` or R's `read.table`.
- Measure every number two ways.

Next: chapter 30 (movies), 31 (VisIt on the cluster), 32 (VisIt from the command line and Python).

REFERENCES AND CREDITS

- L. Waldrop, “Data analysis in VisIt”, MATH 769 (16 February 2015; VisIt 2.5.2, IBAMR data); its 2017 version for IB2d’s `.vtk` output is quoted on slides 4, 5, 14, 22, 23.
- L. A. Miller, “4 – Visit Data Analysis – Sampling and Time Queries” (2019, adapted from Waldrop’s tutorial; pictures reused: slides 11, 13, 15, 21), and “Lecture 9: Sampling, averaging, and analyzing data in Visit” (its title slide: “Lecture 9 – Visit, Sampling and Time Queries”; adapted from Waldrop’s and N. Battista’s tutorials); “3B – Analyzing your IBFE simulation” (2018), slides 12–16.
- VisIt: H. Childs et al., “VisIt: An end-user tool for visualizing and analyzing very large data”, in *High Performance Visualization*, CRC Press (2012) 357–372; version 3.5.0 (Linux), run without a window.
- The runs: `examples/IB/11-heart-tube-pump` (`TARGET_VP`, `FORCE_VP` with `Fmag_vp` $= 0.25$), IBAMR 0.19.0, chapter 9b.
- This chapter’s scripts: `examples/analysis/visit_queries.py`, `read_curve.py`, `read_curve.m`, `templates/`; in the working folder `wsl/final/ch29_*.sh`, `visit_ch29_*.sh` and `tools/ch29_figures.py`.